LAPORAN UJIAN TENGAH SEMESTER

PENGOLAHAN CITRA DIGITAL
IMPLEMENTASI FILTER LoG, SEGMENTASI KARAKTER, DAN PENGHITUNGAN KOIN

KELOMPOK 9

Bambang Herlambang	5803024019
Dustin Pranata Wangsawidjaja	5803024021

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS KATOLIK WIDYA MANDALA

SURABAYA

2026

Daftar Isi

1 Pendahuluan **1**
1.1 Latar belakang dan tujuan . . . 1
1.2 Ruang lingkup dan data . . . 1

2 Penyelesaian Soal Nomor 1: Selisih LoG **2**
2.1 Tujuan dan asumsi . . . 2
2.2 Dasar matematis . . . 2
2.3 Algoritma dan implementasi . . . 3
2.4 Penyimpanan dan hasil pengolahan . . . 3
2.5 Interpretasi sigma kecil dan keterbatasan . . . 4

3 Penyelesaian Soal Nomor 2: Segmentasi Karakter **5**
3.1 Tujuan dan data masukan . . . 5
3.2 Segmentasi dan penggunaan regionprops . . . 5
3.3 Pengurutan dan kode semu . . . 6
3.4 Batasan metode . . . 6
3.5 Hasil dan pemeriksaan . . . 7

4 Penyelesaian Soal Nomor 3: Koin Menurut Ukuran **8**
4.1 Tujuan, asumsi, dan data percobaan . . . 8
4.2 Segmentasi dan pengukuran setiap koin . . . 8
4.3 Aturan pengelompokan dan kode semu . . . 9
4.4 Hasil dan pembahasan . . . 10
4.5 Batasan metode . . . 11

5 Reproduksi dan Pengujian **12**
5.1 Lingkungan eksekusi . . . 12
5.2 Berkas keluaran . . . 12
5.3 Pemeriksaan otomatis . . . 12

6 Kesimpulan **13**

Daftar Pustaka **14**

1 Pendahuluan

1.1 Latar belakang dan tujuan

Pengolahan citra digital memungkinkan perubahan intensitas, pemisahan objek, dan pengukuran bentuk dilakukan secara komputasional. Laporan ini membahas penyelesaian tiga bagian Ujian Tengah Semester mata kuliah Pengolahan Citra Digital, yaitu selisih respons *Laplacian of Gaussian* pada soal nomor 1, segmentasi karakter pada soal nomor 2, serta penghitungan koin menurut ukuran pada soal nomor 3 [1].

Tujuan pekerjaan adalah mengimplementasikan algoritma dalam Python, menjelaskan tahapan dan dasar matematisnya, serta memeriksa hasil dengan data contoh dan pengujian otomatis. Pembahasan nomor 1 mempertahankan standar deviasi yang tertulis pada soal, yaitu 0,1 dan 0,2. Pembahasan nomor 3 menggunakan fungsi `regionprops` untuk mengukur setiap objek koin yang telah dipisahkan.

1.2 Ruang lingkup dan data

Laporan ini mencakup nomor 1, 2, dan 3. Kode, citra masukan, serta hasil perhitungan tersimpan dalam direktori masing-masing soal. Nomor 1 dan 3 dikerjakan oleh Bambang Herlambang (5803024019), sedangkan nomor 2 dikerjakan oleh Dustin Pranata Wangsawidjaja (5803024021).

Tabel 1. Bahan masukan dan implementasi yang digunakan.

Bagian	Bahan dan program
Nomor 1	Citra sintetis grayscale 384×256 piksel pada `soal1/data/citra_uji.png`; program `soal1/main.py`.
Nomor 2	Citra teks pada `soal2/teks.png`; program `soal2/main.py`, dikerjakan oleh Dustin.
Nomor 3	Citra sintetis 1100×1100 piksel berisi 100 koin terpisah pada `soal3/data/koin_100.png`; program `soal3/main.py`.
Validasi	Metadata koin pada `soal3/data/koin_100_ground_truth.json` serta pengujian pada `soal1/tests/` dan `soal3/tests/`.

Citra contoh nomor 1 dan 3 dibuat secara sintetis melalui `buat_data_contoh.py` dengan *seed* 9 agar dapat direproduksi. Citra koin bukan foto uang logam nyata. Data acuan atau *ground truth* hanya dipakai untuk memeriksa hasil; program penghitung membaca citra tanpa mengambil jumlah atau ukuran koin dari metadata acuan.

Kode lengkap dan berkas pendukung tersedia pada repositori GitHub Kelompok 9. Langkah pemasangan dan pelaksanaan disajikan pada Bagian 5.

2 Penyelesaian Soal Nomor 1: Selisih LoG

2.1 Tujuan dan asumsi

Soal nomor 1 meminta pembacaan citra, penerapan filter Laplacian Gaussian dengan standar deviasi 0,1 dan 0,2, serta penyimpanan kedua respons dan selisihnya [1]. Istilah LG diinterpretasikan sebagai *Laplacian of Gaussian* (LoG). Standar deviasi dinyatakan dalam **piksel** pada kedua sumbu, sedangkan selisih didefinisikan sebagai `im1 - im2`. Nilai parameter tersebut dipertahankan sesuai soal.

Masukan RGB diubah menjadi grayscale agar filter bekerja pada dua dimensi spasial. Transparansi RGBA dikompositkan ke putih sebelum konversi warna. Intensitas citra dinyatakan sebagai `float64` dalam rentang $[0, 1]$. Program menolak masukan kosong, dimensi yang tidak didukung, dan intensitas yang tidak berhingga. Data contoh berukuran 384×256 piksel dan tersedia pada `soal1/data/citra_uji.png`.

2.2 Dasar matematis

Gaussian dua dimensi dengan standar deviasi σ dinyatakan sebagai

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \tag{1}$$

Untuk citra intensitas I, respons LoG kontinu adalah

$$L_\sigma = \nabla^2 (G_\sigma * I) = \frac{\partial^2 (G_\sigma * I)}{\partial x^2} + \frac{\partial^2 (G_\sigma * I)}{\partial y^2}, \tag{2}$$

dengan $*$ menyatakan konvolusi. Implementasi diskret menggunakan `scipy.ndimage.gaussian_laplace`, yaitu penjumlahan turunan kedua Gaussian pada setiap sumbu [2]. Perluasan batas memakai `reflect`. Tidak diterapkan perkalian tambahan dengan σ^2 karena normalisasi skala tersebut tidak diminta dalam soal. Keluaran yang dihitung adalah

$$\mathrm{im1} = L_{0.1}, \qquad \mathrm{im2} = L_{0.2}, \qquad D = \mathrm{im1} - \mathrm{im2}. \tag{3}$$

2.3 Algoritma dan implementasi

Program mengikuti empat langkah pada soal sebagai berikut.

a. **Baca citra (image).** Citra dibaca dari berkas masukan, diubah menjadi grayscale bila diperlukan, lalu dinormalisasi sebagai `float64` dalam rentang $[0, 1]$.

b. **Gunakan filter LoG dengan standar deviasi** 0,1**.** Terapkan filter pada citra masukan yang telah disiapkan, lalu simpan respons sebagai `im1`.

c. **Gunakan filter LoG dengan standar deviasi** 0,2**.** Terapkan filter pada citra masukan yang sama, bukan pada `im1`, lalu simpan respons sebagai `im2`.

d. **Tentukan selisih dan simpan sebagai citra hasil.** Program menghitung `hasil=im1-im2` pada array `float64` bertanda sebelum pemetaan tampilan. Keluaran disimpan sebagai `hasil.npy` dan pratinjau citra `hasil.png`.

Langkah a–d dirangkum dalam kode semu berikut.

Kode 1. Kode semu penyelesaian nomor 1.

```
MULAI
    BACA citra
    JIKA RGBA: kompositkan ke latar putih
    JIKA RGB: ubah menjadi grayscale
    KONVERSI intensitas ke float64 dalam [0, 1]
    im1 <- FILTER_LOG(citra, sigma=0.1, batas=reflect)
    im2 <- FILTER_LOG(citra, sigma=0.2, batas=reflect)
    hasil <- im1 - im2
    SIMPAN im1, im2, hasil sebagai array float64
    HITUNG batas tampilan bersama dari ketiga respons
    SIMPAN PNG pratinjau dan metadata parameter
SELESAI
```

Berikut inti fungsi `hitung_selisih_log` pada `soal1/main.py`. Nilai bawaan `sigma_1` dan `sigma_2` adalah 0,1 dan 0,2; pengurangan dilakukan sebelum penyesuaian tampilan.

Kode 2. Perhitungan kedua respons dan selisihnya.

```
1 im1 = gaussian_laplace(citra, sigma=sigma_1, mode="reflect")
2 im2 = gaussian_laplace(citra, sigma=sigma_2, mode="reflect")
3 hasil = im1 - im2
4 return im1, im2, hasil
```

2.4 Penyimpanan dan hasil pengolahan

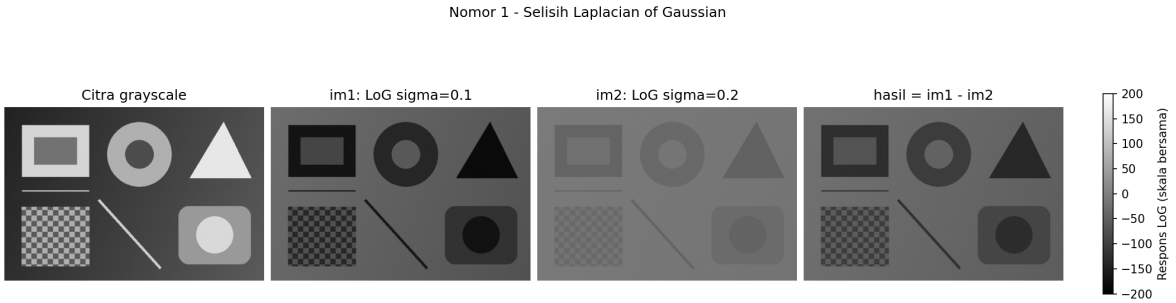
Keluaran utama `im1.npy`, `im2.npy`, dan `hasil.npy` menyimpan array `float64` bertanda. Konversi langsung ke `uint8` sebelum pengurangan dapat menghilangkan informasi negatif. File PNG digunakan untuk pratinjau, sedangkan file `.npy` menjadi acuan perhitungan lanjutan. Semua keluaran bawaan disimpan di `soal1/hasil/`.

Ketiga PNG respons menggunakan rentang bersama $[-M, M]$, dengan M merupakan nilai absolut terbesar dari seluruh piksel ketiga array. Jika ketiganya nol, digunakan $M = 1$.

Pemetaan tampilan dinyatakan sebagai

$$q(F) = \operatorname{round}\left[\frac{255}{2}\left(\operatorname{clip}(F/M, -1, 1) + 1\right)\right]. \tag{4}$$

Nilai nol ditampilkan sebagai abu-abu 128. Nilai negatif lebih gelap dan nilai positif lebih terang. Pemetaan ini tidak mengubah array asli.



Gambar 1. Citra grayscale dan hasil pengolahan nomor 1. Ketiga respons memakai skala bersama, sedangkan citra grayscale memakai rentang $[0, 1]$.

Tabel 2. Rentang respons asli pada citra contoh, dibulatkan enam desimal.

Array	Minimum	Maksimum
`im1`	−200,000000	−18,039216
`im2`	−49,994045	−4,507871
`hasil`	−150,006577	−13,531345

Tabel 2 berasal dari `metadata.json`, yang juga mencatat sigma, ukuran citra, metode batas, dan rentang PNG. Pada data contoh ini M sekitar 200 dan seluruh respons bernilai negatif. Karena skala tampilan dipakai bersama, perbedaan kecerahan antarpanel pada Gambar 1 mencerminkan perbedaan besar respons.

2.5 Interpretasi sigma kecil dan keterbatasan

LoG kontinu ideal memberikan respons nol pada citra konstan. Akan tetapi, sigma 0,1 dan 0,2 jauh lebih kecil daripada jarak antarpiksel. Pada implementasi SciPy 1.15.3 dengan pemotongan kernel bawaan `truncate=4`, sigma 0,1 menghasilkan radius kernel nol. Dalam dua dimensi, respons diskretnya menjadi $L_{0.1}(I) = -2I/(0{,}1)^2 = -200I$, hingga galat pembulatan numerik. Untuk sigma 0,2, respons pada citra konstan $I = c$ adalah sekitar $L_{0.2}(c) = -49{,}9906834c$.

Sebagai pemeriksaan, citra konstan $c = 0{,}4$ menghasilkan `im1` sebesar -80, `im2` sekitar $-19{,}9962734$, dan selisih sekitar $-60{,}0037266$. Selisih yang tidak nol ini merupakan **bias DC akibat sampling kernel**. Oleh karena itu, hasil bawaan dapat menyerupai negatif citra dan tidak ditafsirkan semata-mata sebagai peta tepi. Program tetap menyajikan hasil dengan sigma yang ditentukan soal, disertai array asli dan penjelasan keterbatasan diskretisasinya.

3 Penyelesaian Soal Nomor 2: Segmentasi Karakter

3.1 Tujuan dan data masukan

Bagian ini dikerjakan oleh Dustin Pranata Wangsawidjaja. Program `soal2/main.py` memisahkan karakter pada citra teks menjadi gambar individual dengan bantuan `regionprops` [1, 3]. Tujuannya adalah segmentasi karakter, bukan pengenalan huruf atau OCR. Citra `soal2/teks.png` memuat ZEUS, TWINPEAK, dan ACOSTA, dengan total acuan 18 karakter. Karakter terang berada pada latar gelap, saling terpisah, dan mempunyai orientasi yang bervariasi.

3.2 Segmentasi dan penggunaan regionprops

Citra berwarna diubah menjadi grayscale; masukan satu kanal dinormalisasi dengan membagi intensitas terhadap nilai maksimum. Ambang Otsu menghasilkan mask $B = [I > t]$. Jika lebih dari setengah piksel mask bernilai benar, mask dibalik agar karakter menjadi foreground. Objek dengan luas kurang dari 10 piksel dihapus menggunakan `remove_small_objects`, kemudian `measure.label` memakai konektivitas 8 untuk memberi identitas pada setiap komponen. Nilai luas minimum merupakan parameter implementasi, bukan ketentuan tetap untuk semua resolusi citra.

Tabel 3. Peran atribut regionprops pada segmentasi karakter.

Atribut	Penggunaan
`bbox`	Kotak pembatas untuk penyaringan border dan visualisasi.
`image`	Mask milik region saja untuk gambar karakter individual.
`centroid`	Titik pusat untuk pengelompokan baris dan pengurutan.

Region dipertahankan apabila tinggi kotak pembatasnya kurang dari setengah tinggi citra dan lebarnya kurang dari setengah lebar citra. Penyaringan ini menghapus garis border yang membentuk komponen besar. Aturan tersebut berdasarkan dimensi *bounding box*, bukan berdasarkan luas region.

Mask `region.image` dipakai agar piksel huruf tetangga yang mungkin masuk ke kotak pembatas huruf miring tidak ikut tersimpan. Setiap mask diubah menjadi citra biner 0/255 dan diberi margin hitam tiga piksel. Orientasi huruf dipertahankan; program tidak memutar huruf menjadi tegak.

3.3 Pengurutan dan kode semu

RANSAC dengan `LineModelND` mengelompokkan centroid yang mendekati satu garis. Ambang jarak adalah $0{,}045 \max(H, W)$, dengan dua sampel minimum dan paling banyak 2000 percobaan. Kelompok baris diurutkan menurut rata-rata koordinat vertikal, kemudian karakter pada setiap baris diurutkan dari kiri ke kanan. Sisa satu atau dua objek menjadi satu kelompok terakhir. Prosedur ini mengatur urutan berkas; pemisahan karakter tetap ditentukan oleh komponen terhubung dan `regionprops`.

Kode 3. Alur segmentasi karakter pada program Dustin.

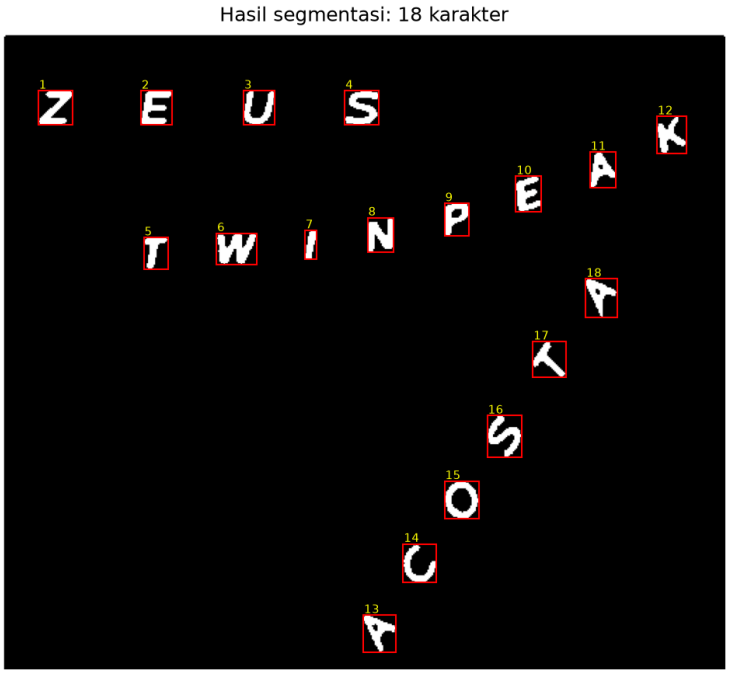
```
INPUT citra teks, luas_min = 10, padding = 3
G = grayscale dan normalisasi citra
B = (G > ambang_otsu(G))
IF rata-rata B > 0.5: B = komplemen B
B = hapus_objek_kecil(B, luas_min)
L = label_8_tetangga(B)
R = regionprops(L)
Pertahankan region dengan tinggi bbox < H/2
dan lebar bbox < W/2
Kelompokkan centroid R ke baris menggunakan RANSAC
Urutkan baris atas-ke-bawah, karakter kiri-ke-kanan
FOR region dalam urutan:
    C = uint8(region.image * 255)
    C = tambah_margin_hitam(C, padding)
    Simpan C sebagai karakter_XX.png
OUTPUT gambar karakter dan visualisasi bbox
```

3.4 Batasan metode

Karakter bersentuhan dapat menjadi satu komponen, sedangkan karakter dengan bagian terpisah, seperti titik dan batang pada huruf i, dapat menjadi beberapa objek. Pengurutan mengasumsikan baris kira-kira lurus dan tidak vertikal. Ambang global dan aturan dimensi border perlu disesuaikan bila latar, resolusi, atau ukuran karakter berubah. Hasil yang dilaporkan berlaku pada citra masukan yang disertakan; program tidak mengklasifikasikan identitas huruf.

3.5 Hasil dan pemeriksaan

Eksekusi ulang menghasilkan 18 karakter, sesuai jumlah yang terlihat pada citra: ZEUS sebanyak 4, TWINPEAK sebanyak 8, dan ACOSTA sebanyak 6. Berkas `karakter_01.png` sampai `karakter_18.png` tersimpan pada `soal2/hasil_karakter/`. Visualisasi kotak pembatas ditunjukkan pada Gambar 2.



Gambar 2. Hasil segmentasi 18 karakter dengan kotak pembatas dan nomor urut.

Pemeriksaan keluaran menunjukkan seluruh 18 gambar merupakan citra biner yang masing-masing memuat satu komponen dengan konektivitas 8. Pada 20 pemanggilan ulang pengurutan terhadap region citra yang sama, urutan label yang diperoleh tetap sama. Ini merupakan pemeriksaan pada data yang disertakan, bukan bukti bahwa pengurutan RANSAC selalu deterministik. Implementasi tidak menetapkan seed acak secara eksplisit.

Tabel 4. Jumlah karakter dan rentang nomor keluaran pada contoh.

Teks acuan	Nomor berkas	Jumlah
ZEUS	01–04	4
TWINPEAK	05–12	8
ACOSTA	13–18	6
	Total	**18**

4 Penyelesaian Soal Nomor 3: Koin Menurut Ukuran

4.1 Tujuan, asumsi, dan data percobaan

Soal nomor 3 meminta penghitungan koin menurut ukurannya dengan bantuan `skimage.measure.regionprops` [1]. Penyelesaian dilakukan melalui segmentasi, pelabelan komponen, pengukuran geometri, dan pengelompokan ukuran. Setiap komponen hasil segmentasi dianggap sebagai satu koin apabila asumsi berikut terpenuhi: koin saling terpisah, terlihat utuh, memiliki kontras terhadap latar seragam, serta mayoritas piksel bingkai citra merupakan latar. Seluruh koin dibandingkan pada skala pengambilan gambar yang sama.

Tidak ada foto koin nyata yang disertakan pada bahan soal yang diterima. Oleh karena itu, demonstrasi menggunakan citra **sintetis** `soal3/data/koin_100.png`, berukuran 1100×1100 piksel, yang dibangkitkan dengan *seed* 9. Citra memuat 100 koin terpisah pada empat radius, yaitu 13, 18, 23, dan 28 piksel, masing-masing 25 buah. Posisi dan detail intensitasnya dibuat bervariasi. Informasi pembangkit disimpan pada `koin_100_ground_truth.json` untuk validasi terpisah; program penghitung hanya membaca citra dan tidak membaca berkas tersebut.

4.2 Segmentasi dan pengukuran setiap koin

Citra masukan dibaca sebagai grayscale atau dikonversi dari RGB ke grayscale. Untuk RGBA, transparansi dikompositkan pada latar putih terlebih dahulu. Intensitas grayscale dinormalisasi ke rentang $[0, 1]$. Citra yang seluruh intensitasnya sama ditolak karena tidak menyediakan pemisah antara objek dan latar.

Ambang t ditentukan menggunakan metode Otsu, yang mencari pemisahan histogram dengan memaksimumkan variansi antarkelas [4]. Mask awal terang didefinisikan sebagai $B(x, y) = [I(x, y) > t]$. Jika sekurangnya setengah piksel bingkai masuk kelas terang, kelas tersebut dianggap sebagai latar dan mask objek menjadi komplemen B. Jika tidak, B langsung dipakai sebagai mask objek. Dengan aturan ini, koin terang pada latar gelap maupun koin gelap pada latar terang dapat diproses.

Lubang tertutup pada mask diisi dengan `binary_fill_holes` agar detail gelap di dalam koin tidak mengurangi luas objek. Komponen diberi label menggunakan konektivitas 8, kemudian komponen dengan luas kurang dari 50 piksel persegi dibuang. Mask yang tersisa dilabel ulang agar setiap koin memiliki ID berurutan. Ambang luas tersebut merupakan parameter `--min-area`, sehingga perlu disesuaikan apabila resolusi citra berubah.

Fungsi `regionprops` menghitung atribut tiap wilayah yang telah berlabel [3]. Fungsi ini tidak menentukan segmentasi atau kelas ukuran secara otomatis. Perannya pada program diringkas dalam Tabel 5.

Tabel 5. Atribut `regionprops` dan penggunaannya.

Atribut	Penggunaan
`label`	ID unik untuk menelusuri objek pada tabel dan anotasi.
`area`	Luas objek A_i dalam piksel persegi.
`equivalent_diameter_area`	Diameter lingkaran yang luasnya sama dengan objek; dasar pengelompokan ukuran.
`centroid`	Pusat objek untuk menempatkan tulisan ID dan kelompok.
`bbox`	Batas kotak objek untuk pemeriksaan lokasi; batas maksimum mengikuti indeks eksklusif.

Ukuran utama yang digunakan adalah diameter ekuivalen:

$$d_i = 2\sqrt{\frac{A_i}{\pi}} = \sqrt{\frac{4A_i}{\pi}}. \tag{5}$$

Besaran ini sesuai untuk objek yang mendekati lingkaran dan dinyatakan dalam piksel. Hasilnya merupakan ukuran geometris pada citra, bukan diameter dalam milimeter atau nilai nominal mata uang.

4.3 Aturan pengelompokan dan kode semu

Diameter diurutkan dari kecil ke besar, dengan ID sebagai pemecah urutan apabila diameternya sama. Untuk setiap objek, program membentuk calon kelompok C dengan menambahkan objek tersebut pada kelompok terakhir. Dengan $m_C = \text{median}\{d_j : j \in C\}$ dan toleransi $\tau = 0{,}08$, penambahan diterima apabila

$$|d_j - m_C| \le \tau m_C + 10^{-12} \quad \text{untuk setiap } j \in C. \tag{6}$$

Jika syarat tidak terpenuhi, objek memulai kelompok baru. Suku sangat kecil 10^{-12} hanya mengantisipasi galat pembulatan numerik. Pemeriksaan terhadap *seluruh* anggota membatasi efek rantai, yaitu penggabungan ukuran yang semakin melebar karena perbandingan hanya dilakukan terhadap objek sebelumnya. Kelompok diberi nama K1, K2, dan seterusnya menurut ukuran menaik. Jumlah kelompok tidak ditetapkan menjadi empat dan jumlah objek tidak dipaksa menjadi 100.

Kode 4. Alur penghitungan koin menurut ukuran.

```
INPUT citra I, luas_min = 50, toleransi = 0.08
G = grayscale dan normalisasi I ke [0, 1]
t = ambang_otsu(G)
B = (G > t)
IF fraksi piksel terang pada bingkai >= 0.5:
    M = komplemen B
ELSE:
    M = B
M = isi_lubang(M)
L = label_8_tetangga(M)
Buang komponen dengan luas < luas_min; label ulang
R = regionprops(L)
Urutkan R berdasarkan diameter ekuivalen, kemudian ID
kelompok = daftar kosong
FOR objek dalam R:
    IF kelompok kosong: buat kelompok pertama
    ELSE:
        C = kelompok terakhir ditambah objek
        m = median diameter seluruh anggota C
        IF semua abs(d - m) <= toleransi*m + 1e-12:
            terima objek dalam kelompok terakhir
        ELSE: buat kelompok baru berisi objek
Hitung anggota setiap kelompok
OUTPUT jumlah, pengukuran, mask, label, dan anotasi
```

Cuplikan berikut memperlihatkan inti pengambilan luas dan diameter dari `regionprops` dalam implementasi `soal3/main.py`. Program lengkap juga menyimpan centroid, kotak pembatas, dan nomor kelompok setiap objek.

Kode 5. Inti pengukuran wilayah berlabel.

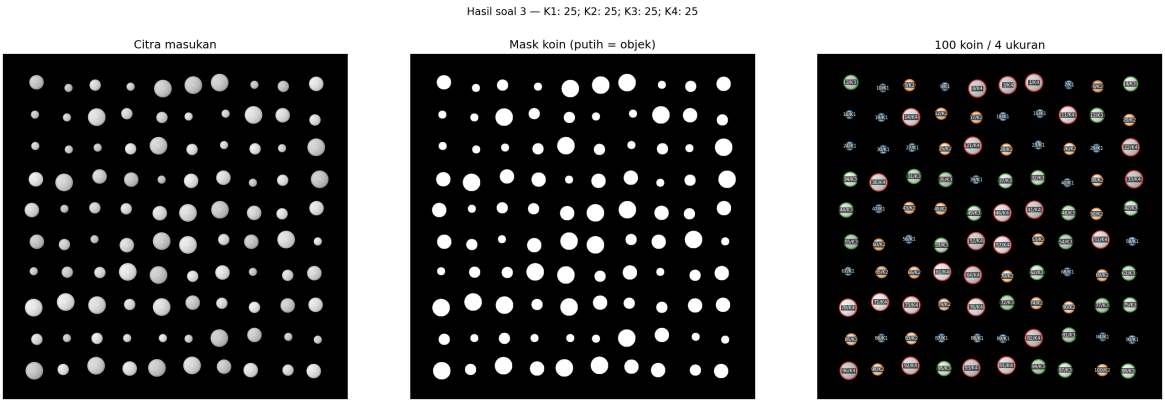
```
objects = []
for region in measure.regionprops(labels):
    objects.append({
        "id_objek": int(region.label),
        "luas_px2": int(region.area),
        "diameter_ekuivalen_px":
            float(region.equivalent_diameter_area),
    })
```

4.4 Hasil dan pembahasan

Pada citra contoh, ambang Otsu pada intensitas ternormalisasi adalah 0,396484375. Bingkai dikenali sebagai latar gelap dan tidak ada komponen yang dibuang oleh penyaringan luas. Hasil pelabelan menghasilkan 100 objek. Pengelompokan dengan toleransi 8% menghasilkan empat ukuran dengan jumlah sebagaimana Tabel 6. Nilai median pada tabel dibulatkan menjadi dua angka desimal untuk penyajian, sedangkan `objek_koin.csv` menyimpan pengukuran tanpa pembulatan penyajian.

Tabel 6. Hasil penghitungan pada citra koin sintetis dengan *seed* 9.

Kelompok	Diameter pembangkit (px)	Median terukur (px)	Jumlah
K1	26	25,85	25
K2	36	36,00	25
K3	46	45,95	25
K4	56	55,93	25
		Total	**100**



Gambar 3. Citra sintetis, mask segmentasi, dan anotasi hasil. Tulisan ID/K menunjukkan identitas objek dan kelompok ukurannya.

Jumlah dan distribusi ukuran sesuai dengan metadata pembangkit yang diperiksa setelah deteksi. Perbedaan kecil antara diameter pembangkit dan median hasil pengukuran timbul karena diskretisasi piksel, penghalusan tepi pada citra sintetis, dan keputusan ambang segmentasi. Angka 100 pada `--expected-count` hanya digunakan untuk mengeluarkan peringatan apabila jumlah hasil berbeda; parameter ini tidak menambah, menghapus, atau membagi objek untuk mencapai jumlah tersebut. Statistik kelompok dan peringatan disimpan pada `ringkasan.json`; mask, label berwarna, serta anotasi menyediakan bukti visual untuk memeriksa kesalahan.

4.5 Batasan metode

Koin bersentuhan dapat bergabung menjadi satu komponen, sedangkan koin terpotong atau tertutup sebagian menghasilkan diameter yang terlalu kecil. Pencahayaan tidak seragam, bayangan, dan latar bertekstur dapat menurunkan mutu ambang global Otsu. Kesalahan pemilihan polaritas juga mungkin terjadi apabila mayoritas bingkai bukan latar. Perbedaan perspektif mengubah skala diameter antarlokasi, sementara dua ukuran fisik yang sangat dekat dapat tergabung akibat toleransi 8%. Karena itu, hasil pada data sintetis ini menunjukkan keberhasilan pada asumsi yang dinyatakan, dan belum membuktikan keandalan pada seluruh foto koin nyata. Parameter luas minimum dan toleransi ukuran tetap perlu diperiksa bersama mask dan anotasi ketika memakai citra lain.

5 Reproduksi dan Pengujian

5.1 Lingkungan eksekusi

Program menggunakan Python 3.10 atau lebih baru. Eksekusi yang diperiksa menggunakan Python 3.10, NumPy 2.2.6, SciPy 1.15.3, scikit-image 0.25.2, Matplotlib 3.10.9, dan Pillow 12.3.0. Semua dependensi program dicantumkan di `requirements.txt`; dependensi pengujian tambahan tercantum pada `requirements-dev.txt`.

Satu cara eksekusi digunakan secara konsisten, yaitu melalui Python di `.venv`. Dari folder utama repositori pada Windows PowerShell, persiapan berikut cukup dilakukan sekali:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Setelah dependensi terpasang, jalankan ketiga program:

```
.\.venv\Scripts\python.exe soal1/main.py
Push-Location soal2
..\.venv\Scripts\python.exe main.py
Pop-Location
.\.venv\Scripts\python.exe soal3/main.py
```

Pemanggilan langsung interpreter tersebut tidak memerlukan aktivasi lingkungan. Folder `.venv` dibuat pada masing-masing komputer dan diabaikan oleh Git.

Nomor 2 dijalankan dari folder `soal2/` karena lokasi masukan dan keluaran mengikuti folder kerja. Tutup jendela visualisasi agar terminal dapat melanjutkan. Kode pekerjaan Dustin dipertahankan.

5.2 Berkas keluaran

Hasil nomor 1 tersimpan di `soal1/hasil/`. Berkas `im1.npy`, `im2.npy`, dan `hasil.npy` menyimpan data numerik bertanda. Berkas PNG merupakan visualisasi, sedangkan `metadata.json` menyimpan parameter dan statistik respons.

Hasil nomor 2 tersimpan di `soal2/hasil_karakter/`: 18 gambar karakter serta `_visualisasi_bbox.png`. Pemeriksaan eksekusi dan hasil nomor 2 dijelaskan pada Bagian 3; tidak termasuk dalam delapan tes otomatis nomor 1 dan 3.

Hasil nomor 3 tersimpan di `soal3/hasil/`. Berkas `mask_koin.png` menunjukkan segmentasi; `label_koin.png` membedakan objek; `anotasi_koin.png` menunjukkan ID dan kelompok. Pengukuran tiap objek tersimpan di `objek_koin.csv`, sedangkan jumlah serta statistik setiap kelompok tersimpan di `ringkasan.json`.

5.3 Pemeriksaan otomatis

Delapan pengujian mencakup tiga pemeriksaan LoG dan lima pemeriksaan penghitungan koin. Pengujian dilakukan dengan:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest -q
```

Tabel 7. Cakupan dan hasil delapan pengujian otomatis.

No.	Pemeriksaan	Hasil
1	LoG pada citra konstan 0,4 dengan $\sigma = 0{,}1$ menghasilkan -80.	Lulus
2	Dua respons dengan sigma sama mempunyai selisih nol persis.	Lulus
3	Impuls pada sigma 1 dan 2 menghasilkan respons bertanda; penyimpanan `.npy` mempertahankan pengurangan.	Lulus
4	Seluruh 100 objek dipasangkan ke pusat acuan; diameter dan kelompok ukurannya sesuai toleransi pemeriksaan.	Lulus
5	Inversi intensitas, sehingga latar menjadi terang, mempertahankan deteksi dan kelompok seluruh koin.	Lulus
6	Resolusi setengah mempertahankan jumlah dan kelas; diameter mengikuti perubahan skala.	Lulus
7	Lubang diisi dan derau kecil dibuang pada contoh sembilan koin; hasil dua kelompok berjumlah tujuh dan dua.	Lulus
8	Citra seragam dan berkas citra kosong ditolak dengan pesan kesalahan.	Lulus

Hasil pelaksanaan adalah `8 passed`. Pengujian pada sigma 1 dan 2 merupakan pemeriksaan tambahan terhadap penyimpanan nilai bertanda; hasil utama nomor 1 tetap memakai sigma 0,1 dan 0,2. Contoh sembilan koin menunjukkan bahwa jumlah hasil tidak dipaksa menjadi 100. Keberhasilan pada data sintetis tidak menyatakan akurasi universal pada foto nyata dengan pencahayaan, perspektif, atau latar yang berbeda.

6 Kesimpulan

Program nomor 1 membaca citra, menghitung dua respons LoG pada $\sigma = 0{,}1$ dan $\sigma = 0{,}2$ piksel, lalu menyimpan selisih `im1 - im2` sebagai data float64 bertanda. Visualisasi dibuat setelah pengurangan, sehingga penyesuaian tampilan tidak mengubah hasil numerik. Sigma yang sangat kecil menyebabkan bias respons pada grid diskret; hal tersebut perlu dibedakan dari perilaku ideal LoG kontinu.

Program nomor 2 memakai komponen terhubung dan `regionprops` untuk memisahkan 18 karakter pada citra contoh menjadi berkas individual. Mask tiap region mempertahankan orientasi karakter dan menghindari penyalinan karakter tetangga di dalam kotak pembatas.

Program nomor 3 memisahkan koin melalui ambang Otsu dan komponen terhubung, mengukur luas serta diameter ekuivalen menggunakan `regionprops`, kemudian menghitung jumlah tiap kelompok ukuran. Pada contoh sintetis, diperoleh 100 koin dalam empat kelompok, masing-masing 25 buah. Delapan pengujian lulus. Penerapan bergantung pada asumsi koin terpisah, terlihat utuh, dan kontras terhadap latar seragam.

Daftar Pustaka

[1] Peter Rhatodirdjo Angka, *UTS INF301 Pengolahan Citra Digital, Semester Gasal 2026/2027*. Program Studi Informatika, Fakultas Teknik, Universitas Katolik Widya Mandala Surabaya, 8 Oktober 2026. Dokumen soal yang diberikan.

[2] SciPy Developers, *scipy.ndimage.gaussian_laplace*. Dokumentasi resmi SciPy. Tersedia pada laman resmi SciPy.

[3] scikit-image Developers, *skimage.measure.regionprops*. Dokumentasi resmi scikit-image. Tersedia pada laman resmi scikit-image.

[4] scikit-image Developers, *skimage.filters.threshold_otsu*. Dokumentasi resmi scikit-image. Tersedia pada laman resmi scikit-image.